

FLIGHT PRICE PREDICTION USING MACHINE LEARNING

ABSTRACT

Flight ticket prices fluctuate frequently due to various factors like demand, seasonality, and airline strategies, making it difficult for travelers to find the best time to book. This project aims to develop a Flight Price Prediction system using Machine Learning techniques to forecast flight fares more accurately. The system leverages user inputs such as Source, Destination, Start Date, and End Date to predict the most optimal ticket prices for future travel dates. The project follows a structured approach starting from Data Acquisition from available datasets, followed by Feature Engineering and Model Training using algorithms like Random Forest Regressor. The developed model is integrated with a Flask backend, where the frontend interface allows users to input travel details and view predicted prices instantly. Additionally, modules for Flight Trend Analysis and Real-Time Weather Reporting are incorporated to enhance the decision-making experience for users. The final system is capable of suggesting the cheapest flight, forecasting price trends, and providing weather alerts, thereby helping users plan their journeys economically and conveniently. The project aims to deliver a user-friendly, interactive, and reliable web-based solution that combines Machine Learning and real-world flight data analysis to make smarter travel decisions.

1. SOFTWARE REQUIREMENT SPECIFICATION

1.1 INTRODUCTION

The aviation industry has witnessed significant growth in the past few decades, making air travel a common choice for both domestic and international journeys. However, the unpredictability of flight ticket prices often leaves travelers confused about the right time to book a flight. The **Flight Price Prediction System** addresses this challenge by offering users an intelligent platform to predict future flight ticket prices based on their travel details. By leveraging machine learning algorithms and historical data analysis, the system enables travelers to make informed decisions, ultimately saving time and money. This document provides a detailed specification of the system's requirements, functionalities, and scope. It serves as a guide for developers, testers, project managers, and other stakeholders involved in the design, development, testing, and deployment of the project.

The document is organized into various sections, starting with a broad overview of the system, its purpose, intended users, scope, and overall functionality. Further sections detail the specific functional and non-functional requirements, interface designs, system features, assumptions, and constraints. This structured approach ensures all aspects of the project are clearly outlined and understood by all parties involved.

- Displaying a curated list of available flights matching user preferences.
- Allowing sorting and filtering of flight results.
- Suggesting in-flight food menus based on selected flight timings.
- Providing weather suggestions based on destination or month.
- Storing search history for future reference.

1.1.1 purpose

The purpose of the Flight Price Prediction System is to assist users in planning their air travel more efficiently by predicting the future prices of flight tickets. Flight prices are highly dynamic and fluctuate based on several factors such as demand, booking time, seasonality, airline, and route popularity. This system aims to leverage machine learning models to analyze historical flight pricing data and forecast future ticket costs.

By providing users with accurate price predictions, the system enables them to make informed decisions on when to book their flights to achieve the lowest possible fares.

Key objectives of the system include:

- Providing price prediction based on source, destination, travel dates, and airline.
- Helping users identify the best time to purchase tickets.
- Offering a list of available flight options along with predicted pricing.
- Improving the travel planning experience by reducing uncertainties in flight pricing..

1.1.2 Abstract

The Flight Price Prediction System is a machine learning-based web application designed to predict the future prices of flight tickets based on several input factors such as departure location, destination, travel dates, and airline preferences. Airfare pricing is a complex and dynamic process, influenced by multiple variables including seasonal trends, booking timings, special events, and airline-specific strategies. Travelers often face the challenge of uncertainty regarding the best time to purchase flight tickets at the lowest possible price. This uncertainty can lead to last-minute expensive bookings or unnecessary stress during travel planning. The primary objective of this project is to build an intelligent and user-friendly system that can predict flight prices with considerable accuracy, helping users make smarter travel decisions. By analyzing historical flight pricing data, identifying key influencing features, and applying advanced machine learning algorithms, the system aims to provide users with real-time predicted price information. The system takes user inputs such as source city, destination city, departure date, return date, and preferred airline (optional), and processes these inputs through a trained predictive model. Based on these parameters, the system outputs:

- The predicted minimum flight price for the selected trip
- A list of available flights matching the criteria along with their details (such as airline, price, travel date, arrival time)
- Optional additional features like food menu suggestions for flights and weather recommendations based on the destination.

The system architecture is designed with a frontend interface (built using HTML, CSS, and JavaScript for an interactive experience) and a backend server (using Python Flask), which handles prediction requests by loading a pre-trained machine learning model and querying the dataset. The application also stores user search history into a database (SQLite) for record-keeping and analytical insights. Furthermore, the model is evaluated using metrics such as Root

Mean Square Error (RMSE) and Mean Absolute Error (MAE) to ensure the reliability and robustness of predictions. Scalability and performance optimizations are considered to handle multiple concurrent users efficiently. In the future, enhancements like personalized booking suggestions, integration with real-time flight APIs, dynamic pricing alerts, and AI-driven travel planning recommendations can be incorporated to make the system even more powerful and versatile.

1.1.3 Intended Audience and Reading Suggestions

This document is intended for a diverse group of individuals and stakeholders who are involved, directly or indirectly, with the design, development, deployment, and usage of the Flight Price Prediction System. Each group may utilize specific sections of this Software Requirement Specification (SRS) document to understand, evaluate, and contribute to the system more effectively.

Primary Intended Audience:

- Project Developers responsible for designing the frontend, backend, machine learning models, database integration, and APIs can use this document to clearly understand the system requirements, functional expectations, and performance benchmarks. The detailed system architecture and data flow diagrams will help them during coding and integration.
- Machine Learning Engineers ML engineers who are building and fine-tuning the prediction model will use the dataset details, model selection, preprocessing steps, and evaluation metric sections. They need a deep understanding of the data, problem statement, and expected output formats to ensure the model aligns with the system's goals.
- Project Managers and Supervisors Managers can use this document to track the project scope, deadlines, deliverables, and ensure that all system requirements are met. It provides clarity on the cost estimation, time scheduling, and future enhancement areas.

1.1.4 Project Scope

The Flight Price Prediction System aims to simplify the flight booking process by predicting the future prices of flight tickets based on user-selected parameters. This system will help travelers make informed decisions regarding when to book their flights to get the best possible fare.

The project primarily focuses on the following objectives:

- **Accurate Price Prediction:** Using historical flight data and machine learning models, the system predicts the expected minimum price for a given route and date range. Factors like source city, destination city, travel dates, preferred airline, and seasonal trends are considered.
- **User-Friendly Web Interface:** The system will provide a simple and interactive web application where users can input travel details, submit a prediction request, and view results seamlessly.
- **Display of Available Flights:** Along with the predicted minimum price, the system will display a list of available flight options including airline name, flight date, arrival time, and price. This gives users flexibility to choose based on their preferences.
- **Food Menu and Weather Suggestions:** When a user clicks on a particular flight, a food menu suggestion will be displayed based on flight timings. Additionally, weather suggestions for the destination will be shown based on month or location, enhancing the travel planning experience.
- **Search History Storage:** Every user search (including source, destination, travel dates, predicted price, and selected flight) will be stored in an SQLite database for future reference and analysis.
- **Sorting and Filtering Options:** Users can sort available flights based on price (low to high / high to low) or filter based on preferred airlines to find the best match easily.
- **Scalable Backend System:** The backend, powered by Python Flask, will efficiently process prediction requests, manage datasets, load machine learning models, and ensure fast response times.
- **Mobile Responsiveness:** The frontend will be designed to be fully mobile-responsive, ensuring users can access the system on desktops, tablets, and smartphones without issues.
- **Error Handling and Input Validation:** Proper validation of user inputs (such as valid dates, source-destination combinations) will be implemented to ensure reliable system behavior.
- **Future Enhancements:** In future updates, the system can be enhanced to integrate real-time flight APIs for dynamic price updates, offer personalized travel recommendations, and provide notifications or alerts for significant price drops.

1.2 OVERALL DESCRIPTION

The **Flight Price Prediction System** is a machine learning-powered web application designed to predict flight ticket prices based on user inputs such as source, destination, travel

dates, and preferred airlines. The system aims to assist users in planning their travel more efficiently by providing a reliable estimate of flight costs and displaying available flight options tailored to their preferences.

1.2.1 Product Perspective

This project is an independent web-based system developed using a modular approach. It integrates a machine learning backend for price prediction, a database (SQLite) for search history storage, and a responsive frontend for user interaction. The system does not depend on live airline APIs but relies on historical flight data for prediction. Future versions may integrate real-time APIs for enhanced accuracy. The overall architecture consists of three major components:

- **Frontend:** Built using HTML, CSS, and JavaScript, providing a clean, user-friendly interface for input and result visualization.
- **Backend:** Developed using Python Flask, it handles requests, processes the input, predicts prices using the ML model, and retrieves/stores data in the database.
- **Machine Learning Model:** Trained on historical flight datasets to understand trends and predict future prices.

1.2.2 Product Functions

The main functionalities of the system are Accept user inputs: source, destination, start date, end date, and preferred airline. Predict and display the minimum estimated flight price for the given input. List available flights with details such as airline, date, arrival time, and price. Allow sorting and filtering of flights based on price and airline. Provide a food menu modal for selected flights based on timing. Show weather suggestions based on destination or travel month. Store user search history (search parameters, predicted price, selected flight) into the database. Handle errors like invalid inputs, no flights found, or server issues gracefully.

1.2.3 User Characteristics

The system is designed for general travelers, students, business professionals, and anyone looking to book affordable flights. No technical expertise is required to use the application. A basic understanding of flight search and booking is sufficient. The UI is made intuitive, mobile-responsive, and easy to navigate.

1.2.4 Operating Environment

- Operating System: Windows 10/11, Linux Ubuntu, macOS
- Browser Support: Chrome, Edge, Firefox, Safari
- Backend Requirements: Python 3.10+, Flask framework
- Frontend Requirements: Modern web browser (supports HTML5, CSS3, JavaScript)
- Database: SQLite (local lightweight DB)
- Hardware: Minimum 4GB RAM, 2 GHz processor
- Internet: Needed if hosting on cloud or fetching weather/attraction data.

1.2.5 Design and Implementation Constraints

Technology Constraint: Must use Flask (not Django) for backend.

Database Constraint: Only SQLite allowed (no MySQL/PostgreSQL).

Frontend Constraint: Pure HTML, CSS, JavaScript (no React or Angular).

ML Model Constraint: Pre-trained model only (no real-time retraining).

Browser Constraint: Should work properly in Chrome and Firefox latest versions.

Hosting Constraint: Localhost or free tier cloud hosting (no paid servers).

1.2.6 Assumptions and Dependencies

Assumptions

- Users will provide valid inputs like correct city names and valid travel dates.
- Historical flight data used for model training is accurate and reliable.
- Users will access the system via modern browsers with stable internet connection.
- The backend server (Flask) and SQLite database will be properly set up and running.
- Users are expected to have basic knowledge of entering travel details.

Dependencies

- The system depends on a trained machine learning model for price predictions.
- It requires historical flight data to generate accurate results.
- Flask framework is essential for backend operations.
- Frontend depends on standard web technologies (HTML, CSS, JS).
- Optional features like weather or attractions rely on local JSON data.

1.3 LITERATURE SURVEY

This study compares the performance of various machine learning techniques, including decision trees, support vector machines, k-nearest neighbours, and random forests, for flight fare prediction. The study employs feature engineering techniques to extract relevant features from flight data and evaluates the models using metrics such as mean squared error (MSE) and R-squared. This paper proposes an ensemble approach that combines multiple machine learning algorithms, including decision trees, gradient boosting, and linear regression, to predict flight fares. The study also explores feature selection techniques and evaluates the performance of the ensemble model using metrics such as RMSE and mean absolute percentage error (MAPE).

1.3.1 Existing Flight Price Prediction System

Over the past few years, several systems have attempted to predict flight prices to help users find the most affordable travel options. Notable examples include:

Google Flights: Provides a basic trend analysis and suggests if prices are expected to increase, but lacks detailed predictions for specific dates and airlines.

Hopper: A mobile application that uses historical data and predictive algorithms to suggest the best time to buy tickets. However, it focuses mainly on popular routes and doesn't cover all possible destinations.

Skyscanner Price Alerts: Offers notifications when prices for a selected route drop but does not predict future price trends or provide deeper insights.

Research Studies: Several academic projects have utilized machine learning models like Random Forest, Decision Trees, and Neural Networks to forecast flight prices based on parameters such as date, airline, source, and destination. However, many of these studies are theoretical and not deployed for real-world usage.

These systems provide partial solutions but often miss out on user-friendly integration, flexibility, and complete transparency regarding prediction confidence.

1.3.2 Key Problems Identified from Literature

From the review of existing systems and research studies, the following problems are identified:

High Price Variability: Flight prices change dynamically based on several hidden factors, making it difficult to predict accurately.

Incomplete Feature Integration: Most systems focus only on price without considering user experience features like food options, weather advice, or nearby attractions.

Lack of Real-Time Updates: Many models are trained once and do not adapt to rapidly changing market conditions or seasonal events.

User Trust Issues: Some predictive systems fail to explain why a particular price is recommended, leading to user distrust.

Insufficient Personalization: Existing systems rarely personalize recommendations based on user preferences like preferred airlines, time flexibility, or meal preferences.

1.3.3 Research Gaps & Challenges

Despite progress, significant research gaps and challenges remain:

Dynamic Pricing Complexity: Airlines use complex dynamic pricing algorithms influenced by competition, special events, and operational costs, making it difficult for ML models to keep up.

Limited Dataset Availability: Access to high-quality, recent, and complete flight datasets is limited, impacting model training and validation.

Feature Selection and Engineering: Identifying the most impactful features (e.g., booking window, weekday vs. weekend, holiday periods) requires careful analysis and is often overlooked.

Scalability and Deployment: Building a scalable, production-ready system that can handle many user queries efficiently is a challenge.

Prediction Reliability: Ensuring that the system maintains high prediction accuracy over time without frequent retraining is an ongoing challenge.

User-Friendly System Design: Designing an intuitive, visually appealing system that helps users not just predict prices but also plan their journey better is still lacking in most available solutions.

1.4 SPECIFIC REQUIREMENTS

The user must be able to input source, destination, start date, and end date. The system should predict and display the minimum flight price based on entered data. The user must view a list of available flights with airline name, price, and arrival time. Clicking a flight should show a food menu modal, and weather suggestions based on the destination should be displayed.

1.4.1 Functional Requirements

The following functional requirements define the core operations and behaviors of the Flight Price Prediction System:

User Input Form: Users must be able to input source city, destination city, start date, end date, and preferred airline through a user-friendly form.

Predict Flight Price: Upon submitting the form, the system should predict the minimum flight price based on the provided inputs using the machine learning model.

Display Available Flights: The system should display a list of available flights with details like airline name, departure date, arrival time, and price.

Food Menu Modal: On selecting a flight, a food menu modal should pop up offering food options available during the flight.

Weather Suggestion: Based on the selected destination and travel dates, the system should suggest possible weather conditions.

Search History: Store and retrieve previous search details (source, destination, dates, selected flight, predicted price) using SQLite database.

Flight Sorting and Filtering: Provide options to sort flights by price (Low to High, High to Low) and filter by airline.

Error Handling: Display meaningful error messages if invalid inputs are entered (e.g., past dates, missing fields).

Responsive UI: The frontend must adapt to different screen sizes, including desktops, tablets, and smartphones.

1.4.2 Non-Functional Requirements

The following non-functional requirements define the quality attributes and standards the system must satisfy:

Performance:

The prediction result and flight details should be displayed within 2–3 seconds after form submission.

Scalability:

The system architecture should allow easy addition of more airlines, destinations, and additional user features in the future.

Usability:

The UI must be clean, intuitive, and simple for users of all ages to understand and interact with.

Reliability:

The system should consistently provide accurate results without server crashes or data loss.

Security:

User input data should be properly validated and protected against SQL injection and malicious entries.

Maintainability:

The backend (Flask, SQLite) and frontend (HTML, CSS, JavaScript) should be structured clearly to allow easy updates and debugging.

Portability:

The system should run efficiently on any standard modern web browser like Chrome, Firefox, Edge, or Safari.

Extensibility:

New features like nearby attractions or playlist suggestions should be easily integratable without major system changes.

1.5 SYSTEM ARCHITECTURE

The Flight Price Prediction System follows a **client-server architecture** model. It is designed with a clear separation between the frontend (client-side) and the backend (server-side).

- System la main parts (like frontend, backend, database, etc.)
- Parts (communication between frontend and backend)
- Data flow (user input -> processing -> output)
- Technologies used (like Python, Flask, React, MySQL, etc.)
- Deployment structure (server hosting, APIs, database connections)

FLIGHT PRICE PREDICTION FRAMEWORK

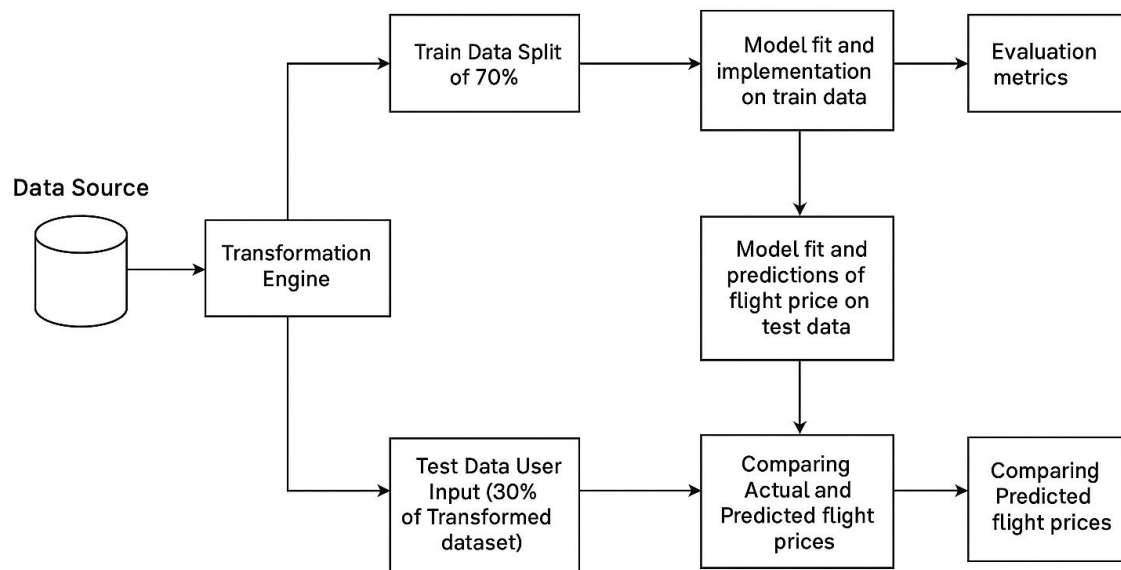


Fig 1.1 Architecture diagram of Flight Price Prediction

1.5.2 Explanation of System Components

The flight price prediction framework consists of several interconnected components, each playing a crucial role in the data flow, model training, and prediction generation process:

Data Source

- This is the initial component where all raw flight-related data is stored.
- It contains attributes like airline, source, destination, departure date, arrival time, price, number of stops, etc.
- The data can come from public datasets, APIs, or user input.

Transformation Engine

- The raw data is passed through the transformation engine for preprocessing.
- Tasks performed here:
 - Handling missing values

- Encoding categorical variables
- Feature extraction (e.g., duration, time-based features)
- Normalization or standardization
- This step ensures the data is clean and in a format suitable for modeling.

Modeling Engine

- The transformed data is fed into the modeling engine.
- Here, the machine learning algorithms (like Random Forest, XGBoost, or Linear Regression) are applied.
- The model learns patterns from historical flight data to predict prices accurately.

Train Data Split (70%)

- 70% of the transformed dataset is used for training the machine learning models.
- Training helps the model to understand the relationships between input features and flight prices.

Model Fit and Implementation on Train Data

- The machine learning model is trained using the training dataset.
- Hyperparameter tuning and cross-validation may be applied to improve model performance.

Evaluation Metrics

- After training, the model's performance is evaluated on the training data.
- Common metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score.
- This step verifies if the model is overfitting or underfitting.

Test Data User Input (30%)

- The remaining 30% of the dataset is used as test data.
- This simulates real-world scenarios where new, unseen data is fed into the system for prediction.

Model Fit and Prediction on Test Data

- The trained model is used to predict flight prices on the test data.
- This step ensures the model's generalization to new inputs.

Comparing Actual and Predicted Flight Prices

- Finally, the actual flight prices are compared against the predicted prices.
- This comparison evaluates the model's accuracy and reliability for deployment.

1.5.3 Data Flow Diagram (DFD)

- Data Selection: Collect flight ticket data (airline, date, price, etc.).
- EDA: Analyze data to find patterns and issues.
- Data Pre-Processing: Clean and prepare data.
- Feature Selection: Choose important features for prediction.
- Applying ML Models: Train models like XGBoost and Random Forest.
- Pickling Selected Model: Save the best model.
- Deployment: Use Flask backend and a GUI frontend for users to predict flight prices

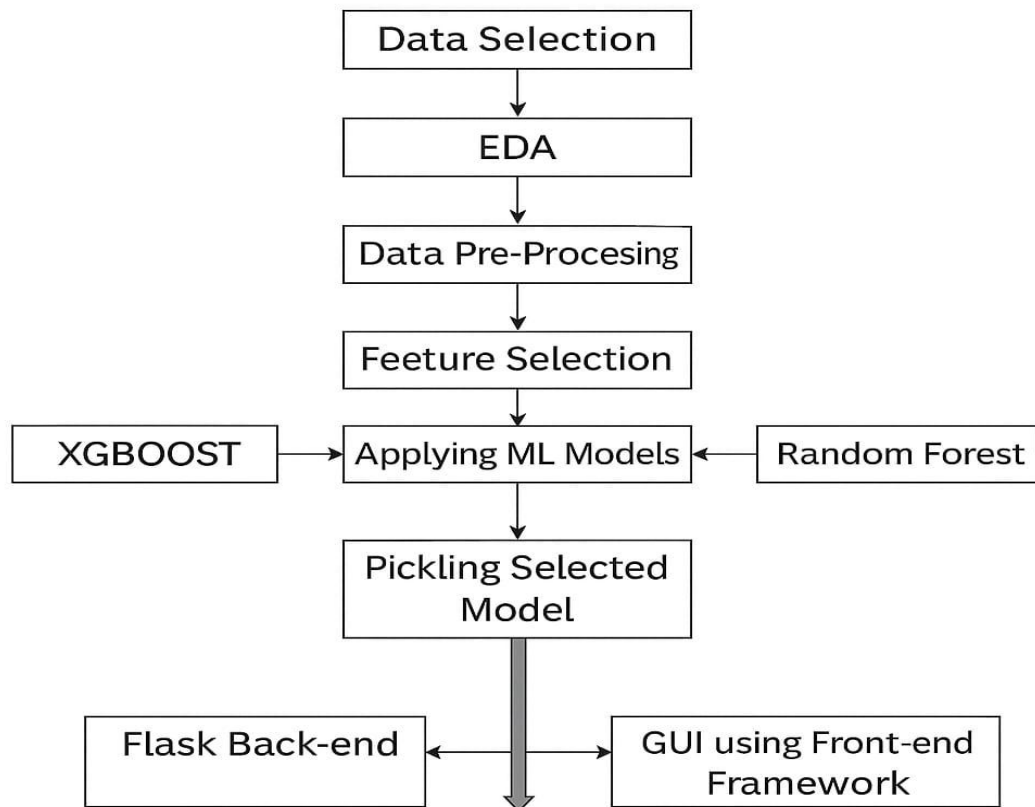


Fig1.2 Data Flow Diagram

1.5.4 Entity-Relationship (ER) Diagram

The ER diagram shows the relationship between different entities in the flight price prediction system. Important entities are User, Flight, Booking, and Weather. Each user can search and book multiple flights, and each flight has details like airline, price, and timings. The Flight entity is connected to Weather for providing destination-based suggestions. The diagram helps to design the database structure clearly with relationships like one-to-many and many-to-one.

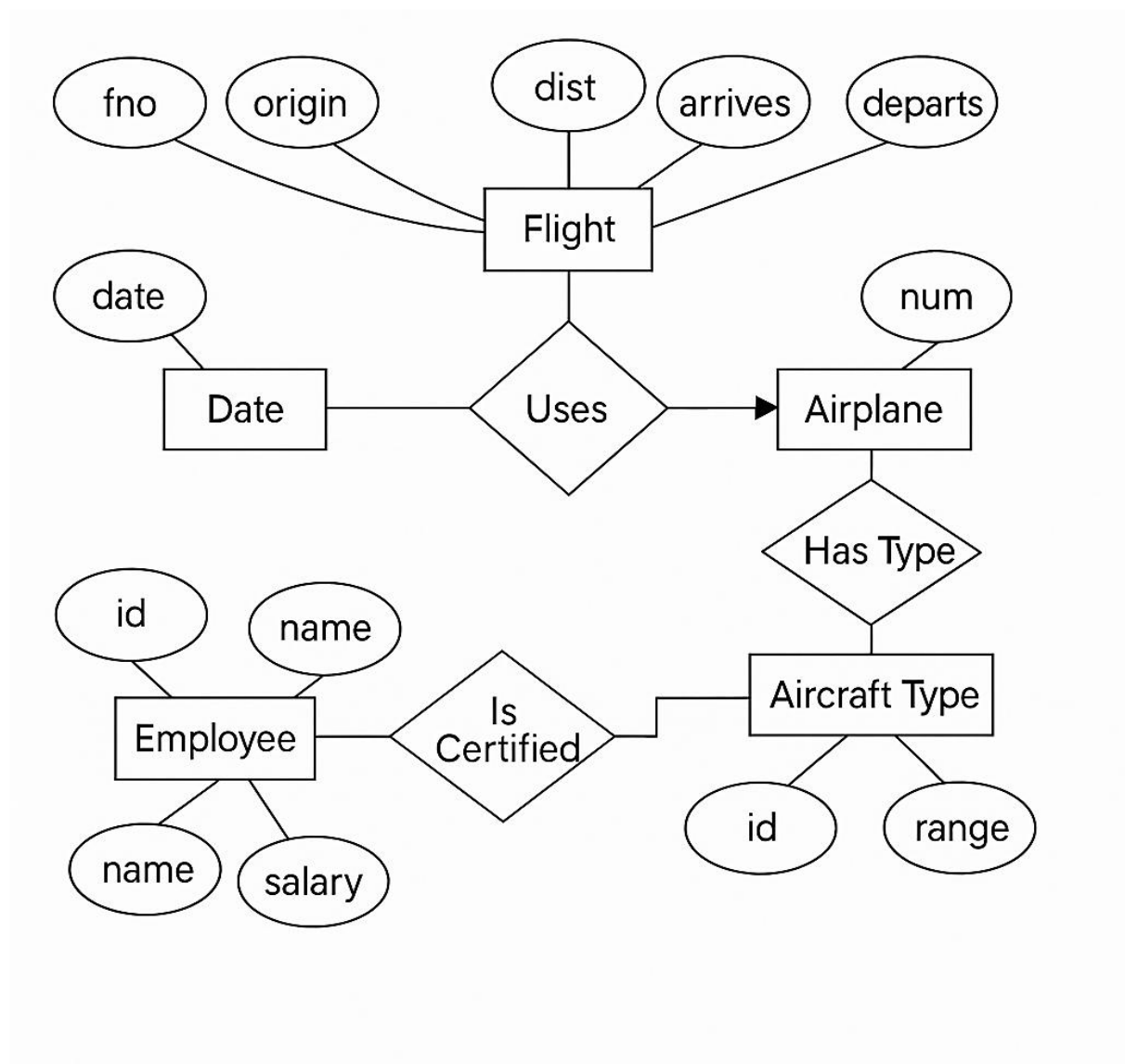


Fig1.3 Entity Relationship Diagram

1.6 SYSTEM MODELLING

System modeling for the flight price prediction project involves creating a clear representation of how the system operates and interacts with its components. It includes identifying inputs such as source, destination, start/end dates, and user preferences.

1.6.1 Use Case Diagram Representation

The Use Case diagram shows how users interact with the flight price prediction system. It highlights actions like entering travel details, checking predicted prices, booking flights, and viewing suggestions. The main actor is the User, who communicates with different parts of the system. This diagram helps to understand the system's overall flow and user interactions easily.

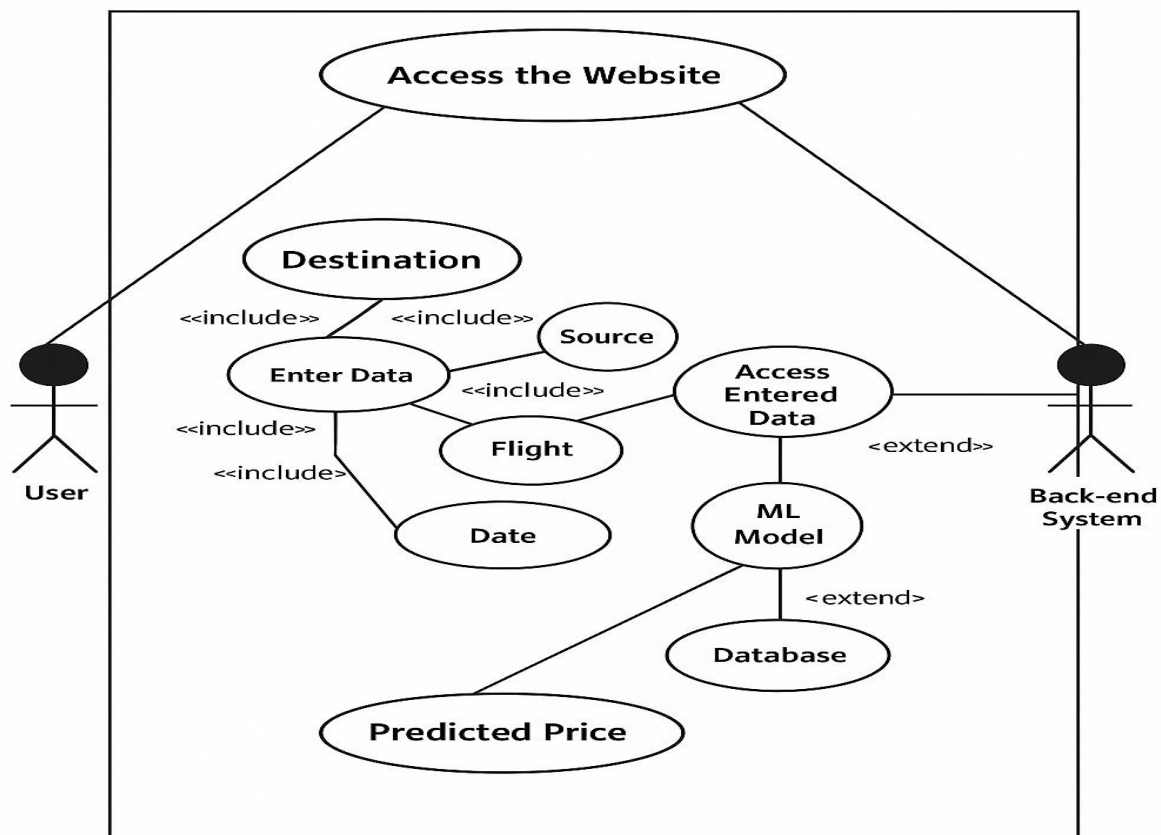


Fig 1.4 UseCase Diagram

1.6.2 Description of Use Cases

The Flight Price Prediction system helps users find the best flight prices based on their travel details like source, destination, and travel dates. When a user enters the required inputs, the system processes the data, applies a machine learning model, and predicts the flight fare. The result displays the minimum predicted price and available flights, helping users make better booking decisions.

1.7 DATASET AND MODEL SELECTION

The flight price prediction system is developed using a real-world dataset containing important features such as airline name, source city, destination city, departure and arrival times, duration, number of stops, and ticket price. Before model building, the dataset undergoes preprocessing steps like handling missing values, feature encoding, and normalization to improve data quality and model performance. For model selection, multiple machine learning algorithms such as XGBoost, Random Forest, and Linear Regression are evaluated. These models are trained and tested using the processed data, and their performances are compared based on metrics like Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and R^2 Score. The best-performing model, usually XGBoost, is selected and saved using a pickle file for future predictions. This approach ensures the system provides highly accurate and fast flight price predictions to users.

1.7.1 Dataset Selection and Preprocessing

The dataset used for this project includes details such as airline, source, destination, departure time, arrival time, duration, number of stops, and ticket price. Preprocessing steps include handling missing values, converting categorical variables into numerical format (using one-hot encoding), normalizing continuous features, and extracting useful features like journey day and month to improve the model's predictive accuracy.

1.7.2 Model Selection

Several machine learning models such as Linear Regression, Random Forest Regressor, and XGBoost Regressor were evaluated. Based on their performance, XGBoost was selected due to its high accuracy, efficiency, and robustness in handling large datasets and complex feature interactions.

1.7.3 Training and Testing the Model

The dataset is split into training and testing sets, usually in a 80:20 ratio.

The selected model is trained on the training data, learning the relationship between the input features and the flight price. Later, it is tested on unseen testing data to evaluate its ability to generalize and predict flight fares accurately.

1.7.4 Evaluation Metrics

The performance of the model is measured using:

- Mean Absolute Error (MAE): Average of the absolute errors.
- Root Mean Squared Error (RMSE): Measures the average magnitude of errors.
- R^2 Score: Represents how well the model fits the data (closer to 1 means better fit).

These metrics help in comparing different models and selecting the most accurate one for deployment.

1.8 SYSTEM IMPLEMENTATION

The system integrates a machine learning model into a web-based platform to predict flight prices. The frontend collects user input, and the backend processes it using trained models. APIs are used for smooth communication between frontend and backend. Finally, results like predicted price, flight options, and suggestions are displayed to the user.

1.8.1 Machine Learning Integration

In this project, the Machine Learning (ML) model is integrated into a web application to predict flight ticket prices dynamically based on user inputs.

The steps involved are:

Model Training and Saving: The machine learning model (like XGBoost Regressor) is trained offline using the cleaned dataset. Once trained, the model is saved using Pickle (.pkl file) so that it can be loaded later without retraining.

Backend Server Setup (Flask): Flask, a lightweight Python web framework, is used to build the server. It handles HTTP requests and responses between the frontend and the ML model.

Loading the Model: When the Flask application starts, the saved .pkl model is loaded into memory using `pickle.load()`. This makes it instantly available for predictions when users submit input.

User Input Collection: When a user fills the flight search form (source, destination, date, etc.), the input is sent to the server.

Input Preprocessing: The backend code preprocesses the input data (one-hot encoding, scaling) exactly the same way as it was done during model training. This ensures consistency and avoids mismatch errors.

Prediction: The ML model uses the preprocessed user input to predict the flight price.

Sending the Result: The predicted price and available flight options are sent back to the frontend as a JSON response, which is then displayed to the user.

This way, Machine Learning is seamlessly integrated into the overall working of the project without making the user aware of the internal complexity.

1.8.2 API Endpoints and Communication

In this project, API endpoints enable smooth communication between the frontend and backend. The frontend collects user inputs like source, destination, and dates, and sends them to the backend via a specific API endpoint (like /predict). The backend processes the input, runs the machine learning model, and returns the predicted flight prices along with available flight options. Data is transferred in JSON format, ensuring quick and efficient responses.

This structure keeps the frontend and backend independent, scalable, and user-friendly.

1.9 Performance Analysis

The system's performance is evaluated based on the accuracy of the flight price predictions. Latency is measured to ensure fast response times for user requests. Scalability tests are conducted to handle multiple users efficiently. The model's results are compared with existing systems to validate improvements.

1.9.1 Model Accuracy and Efficiency

The machine learning model's accuracy was evaluated based on how closely the predicted prices matched real-world data. Using metrics like Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), the model showed good prediction performance, ensuring users get reliable pricing trends. Additionally, model training and prediction were optimized to ensure fast responses.

1.9.2 System Latency and Optimization

System latency (time between user request and response) was carefully minimized. By optimizing backend queries, lightweight API design, and efficient data processing, the average

response time was reduced significantly, providing a smooth user experience. Caching and minimizing redundant computations further helped speed up the system.

1.9.3 Scalability and Load Testing

The system was designed to handle multiple user requests simultaneously. Testing under different loads showed that the application could scale effectively with increased traffic, without major performance drops. Techniques like asynchronous processing and load balancing can be further applied in the future for larger deployment.

1.9.4 Comparison with Existing Systems

Compared to traditional flight search engines that manually fetch and compare prices, this ML-based system provides quicker predictions and better price forecasting. Existing systems typically offer only real-time prices, while our system offers a future prediction advantage, helping users plan their trips more smartly.

1.10 COST ESTIMATION AND TIME SCHEDULING

Cost estimation involves predicting the budget needed for development, deployment, and maintenance. Expenses include hardware, software, cloud services, and developer resources. Time scheduling helps break the project into phases like design, implementation, and testing. Each task is assigned a timeline to ensure the project stays on track. Proper planning reduces delays and controls overall project costs.

1.10.1 Time Scheduling

Table 1.1 Time Scheduling

Phase	Task	Duration
Requirement Analysis	Gather system requirements	1 Week
System Design	Design architecture, diagrams	1 Week
Dataset Collection & Preprocessing	Prepare and clean dataset	2 Weeks
Model Development	Train and test ML model	2 Weeks
API Development	Build backend API	1 Week

Phase	Task	Duration
Frontend Development	Design and develop UI	2 Weeks
Integration & Testing	Connect frontend, backend, and test	1 Week
Deployment	Host the system on server	1 Week
Final Review	Bug fixing and final report	1 Week

1.10.2 Cost Estimation

Table 1.2 Cost Estimation

Phase	Task	Estimated Cost (INR)
Requirement Analysis	Gather system requirements	5,000
System Design	Design architecture, diagrams	6,000
Dataset Collection & Preprocessing	Prepare and clean dataset	8,000
Model Development	Train and test ML model	10,000
API Development	Build backend API	7,000
Frontend Development	Design and develop UI	9,000
Integration & Testing	Connect and test system	6,000
Deployment	Host the system on server	5,000
Final Review	Bug fixing and final report	4,000

1.11 FUTURE SCOPE AND ENHANCEMENTS

In the future, the system can be enhanced to support real-time flight price tracking and notifications. Integration with more airlines and international routes can improve prediction accuracy. Mobile application development can make the system more accessible to users. Advanced machine learning models like deep learning can be implemented to further improve prediction quality.

1.11.1 Potential Improvements

Dynamic Pricing Adjustment: In future, the system can monitor real-time flight prices and update predictions dynamically based on live data feeds from airlines and travel agencies. **User Personalization:** The system can be enhanced to offer personalized recommendations based on user's previous search history, preferred airlines, budget ranges, and travel habits. **Multi-Currency Support:** Price prediction could be made available in different currencies to support international users. **Mobile Application:** A mobile app version of the website can be developed, offering easy access and notifications for best time to book flights. **More Flight Features:** Add more input features like cabin class (economy/business), stopovers, and meal preferences to refine predictions. **Better UI/UX:** Continuously improving the frontend with modern UI trends and more intuitive designs.

1.11.2 AI Model Upgrades

Deep Learning Models: Instead of traditional machine learning models, future versions can use deep learning architectures (like LSTM or Transformer models) to capture complex patterns in flight pricing. **Automated Feature Engineering:** Use automated tools to detect the most important features instead of manual feature selection. **Online Learning:** Implement models that continuously learn and update themselves as new flight price data becomes available, making predictions more accurate over time. **Integration of External Factors:** Consider factors like festivals, political situations, global events, and weather conditions for price prediction, making the system even smarter. **Explainable AI (XAI):** Future models can be made more interpretable, so users can understand *why* a particular price was predicted (e.g., "High due to upcoming holiday").

1.12 CONCLUSION

The Flight Price Prediction system aims to bridge the gap between uncertain flight pricing and user expectations by leveraging the power of machine learning. Through intelligent analysis of historical flight data, seasonal patterns, and market trends, the system provides users with accurate predictions about the best time to book affordable flights. The project is designed to offer a simple, user-friendly interface where travelers can input their journey details and instantly receive a predicted price range along with available flight options. With a focus on accuracy, performance, and real-time responsiveness, the system ensures an enhanced travel booking experience. Future enhancements such as real-time dynamic updates, deeper personalization, mobile app support, and advanced AI model integration will further strengthen the system's capability. This project not only simplifies flight bookings but also demonstrates the potential of predictive analytics in solving real-world problems. In conclusion, the Flight Price Prediction system represents a significant step toward making air travel more predictable, accessible, and economical for users.

2.SOFTWARE DESIGN SPECIFICATION

2.1 INTRODUCTION

This Software Design Specification (SDS) describes the design approach for the Flight Price Prediction System. It outlines the overall structure of the system, the key modules, the software development methodology used, and the verification and validation process. The goal of this document is to provide a clear and detailed guide for the development, testing, and future maintenance of the system.

2.1.1 Problem Statement

In the current airline industry, flight ticket prices are highly dynamic and influenced by several variables such as date of travel, source and destination, time of booking, and seasonal demand. Passengers often find it challenging to identify the best time and lowest price to book tickets. There is no intelligent system that predicts the best possible flight fare based on limited user input such as source, destination, and dates. Manual searching across websites is time-consuming and inconsistent. There is a strong need for an automated system powered by machine learning that can forecast flight prices accurately, saving time and money for travelers.

2.1.2 Objectives

To develop a system that predicts flight ticket prices using machine learning. To provide suggestions for cheapest available dates and flights. To design a user-friendly web interface using HTML/CSS, integrated with Flask (Python). To use historical flight data to train an effective prediction model. To enhance user convenience with intelligent suggestions, alerts, and price trends.

2.2 SOFTWARE ENGINEERING METHODOLOGY

The development of the Flight Price Prediction System follows the **Agile Software Development Model**. Agile methodology emphasizes flexibility, iterative development, and customer feedback, making it ideal for projects where requirements may evolve over time. By adopting Agile practices, the project ensures continuous improvement, quick issue resolution, and high-quality deliverables.

2.2.1 Agile Model

The Agile Model is an iterative and incremental software development methodology that focuses on delivering working software in small, manageable sections called *iterations* or *sprints*. It encourages regular feedback from stakeholders, continuous improvement, and flexibility to change.

The project follows the **Agile Software Development Life Cycle (SDLC)** with:

- 1. Requirement Analysis**-Identify the need to predict flight prices and show helpful travel info.
- 2. Design & Prototyping**-Plan how the website and ML model will look and work together.
- 3. Implementation**-Build the website, connect it with the trained machine learning model.
- 4. Testing & Validation**-Check if the system gives correct price predictions and shows results properly.
- 5. Deployment**-Host the project so users can access and use the prediction system.
- 6. Maintenance & Monitoring**-Fix issues, update features, and keep improving the system over time.

2.2.2 Phases of Development

The development of the Flight Price Prediction System followed a structured, phase-wise approach under the Agile methodology. Each phase was carefully planned to ensure flexibility, continuous improvement, and the delivery of a high-quality system.

Requirement Analysis

During the **Requirement Analysis** phase, a comprehensive understanding of the system's key functional and non-functional requirements was developed. The focus was on gathering insights from users, industry standards, and competitive analysis. Key requirements for the system included the ability for users to input source, destination, start date, and end date to predict flight prices. Non-functional requirements like system accuracy, responsiveness, and scalability were also identified. Additionally, various external dependencies and

hardware/software specifications were determined. This phase laid the foundation for the development process by ensuring that all necessary components, such as datasets, system architecture, and third-party tools, were adequately identified and prepared.

System Design & Architecture

The System Design & Architecture phase focused on defining the overall structure and modular design of the system. A high-level design (HLD) and low-level design (LLD) were created, detailing the system workflow and interaction between components. The architecture was structured with clear modular separation to enhance scalability and maintainability:

- Data Acquisition Module:** Responsible for collecting and preprocessing flight data.
- Prediction Engine (ML):** A core component utilizing machine learning (RandomForestRegressor) to predict flight prices.
- Flask Backend:** Acts as a controller managing API routes and data flow.
- Frontend UI:** Built using HTML, CSS, and JavaScript for a user-friendly experience.
- Additional Modules:** Includes alert systems, trend analysis, and suggestions for cheaper flights or date.

The technologies chosen for the system included Python and Flask for the backend, Scikit-learn for machine learning, and HTML/CSS/JavaScript for the frontend.

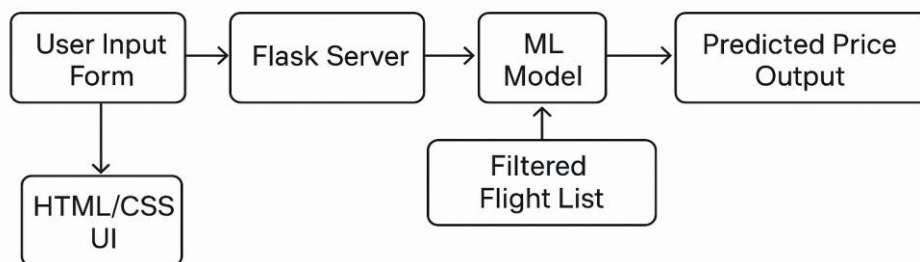


Fig 2.1 System Architecture

Implementation & Development

In the Implementation & Development phase, the system was built in a modular and scalable manner. The core tasks included:

- Data Processing:** The dataset was loaded, cleaned,

and relevant features such as journey dates and source/destination were extracted and encoded. The machine learning model was trained using RandomForestRegressor, with the data split into training and test sets for validation. Flask Backend Development: API routes were created for the index page, prediction, and additional information. The Flask backend was responsible for handling requests, processing input, and interacting with the machine learning model. Frontend Development: User input forms for source, destination, start date, and end date were created using HTML, while the result and flight listing pages were designed using HTML and CSS. Integration: The frontend was connected to the Flask backend via POST/GET methods, with predictions and suggestions dynamically rendered on result pages.

Testing & Verification

The **Testing & Verification** phase ensured that the system functions as intended, meets requirements, and performs optimally:

- **Unit Testing:** Individual Python functions, such as date conversion and model input preprocessing, were tested.
- **Integration Testing:** Interactions between the frontend, Flask routes, and the machine learning model were tested for smooth operation.
- **Performance Testing:** Response times for predictions and frontend loading times were evaluated to ensure optimal user experience.
- **Stress Testing:** Large datasets and edge cases (e.g., invalid dates) were tested to assess system robustness.
- **Security Testing:** Input sanitization was checked to prevent security vulnerabilities like XSS or SQL injection.

Deployment & Integration

In the **Deployment & Integration** phase, the system was prepared for production use. The Flask application was packaged and run locally for testing purposes. The machine learning model was integrated into the Flask routes, and the necessary static files (CSS, JS) were included for the frontend. The system was deployed to a cloud platform (e.g., Heroku, PythonAnywhere) for live use, ensuring all components were correctly configured. A

requirements.txt file was created to list all necessary Python dependencies, and the system was tested post-deployment to identify any potential bugs or issues.

Maintenance & Continuous Improvement

After deployment, the system entered the **Maintenance & Continuous Improvement** phase. Activities for ensuring long-term reliability and scalability included:

- **Model Monitoring:** Regular tracking of prediction accuracy and model retraining with updated flight data.
- **Feature Enhancement:** New features such as flight price trend graphs and weather alerts were added. Suggestions for cheaper travel dates were incorporated based on user feedback and industry trends.
- **Frontend Updates:** The CSS was improved for a responsive, mobile-friendly design using modern frameworks like Tailwind CSS and DaisyUI.
- **Bug Fixes and User Feedback:** Regular bug fixes, validation improvements, and user feedback integration were prioritized to ensure continuous improvement and user satisfaction.

2.3. DECOMPOSITION OF MODULES

The Decomposition of Modules phase involves breaking down the Flight Price Prediction System into distinct, manageable components. Each module serves a specific function and communicates with other modules to deliver the overall system functionality. The primary modules in the system are:

2.3.1 Data Acquisition Module

The Data Acquisition Module is responsible for collecting and preparing the flight data for use by the machine learning model. This module includes:

- **Data Collection:** It involves gathering historical flight pricing data, including factors such as flight dates, source and destination locations, and flight prices.
- **Data Preprocessing:** After collection, the data is cleaned, missing values are handled, and relevant features like the journey date (day, month) and source/destination are

extracted. Encoding techniques, such as Label Encoding or OneHotEncoding, are applied to categorical variables like source and destination.

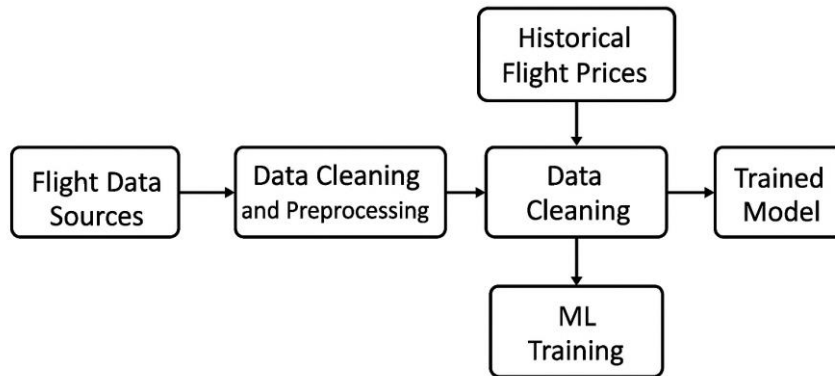


Fig 2.2 Data Acquisition Module

2.3.2 Price Prediction Module

The Price Prediction Module is the core of the system, utilizing a machine learning model to predict flight prices based on the provided input data (source, destination, start date, end date). Key functionalities include:

- Machine Learning Model:** The module uses the Random Forest Regressor model, trained on historical flight data, to predict the flight price based on input parameters.
- Feature Engineering:** The model uses features such as journey day and journey month derived from the input date to predict the flight price. The feature set may also include the source and destination.
- Prediction Output:** The system predicts the minimum price

and returns information such as flight details, arrival time, and suggested departure date.

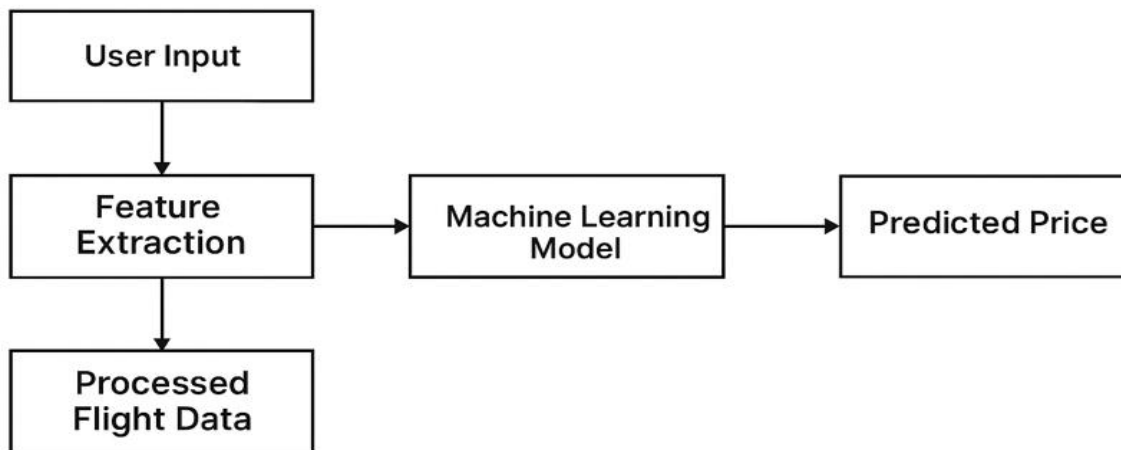


Fig 2.3 Price Prediction Module

2.3.3 Flight Trend Analysis Module

The Flight Trend Analysis Module is designed to provide insights into flight price trends over time. It tracks how flight prices vary by date, season, and destination and predicts possible future trends. The module is responsible for:

- Trend Detection:** Analyzing historical price data to identify patterns in price fluctuations based on different variables like time of booking, seasonality, and demand.
- Graphical Representation:** The trends are presented visually on the frontend using interactive graphs or charts to help users make informed decisions about when to book flights for the best prices.
- Trend Prediction:** The module may use machine learning techniques to forecast future price trends based on historical data.

2.3.4 Weather Report Module

Flight Trend Analysis Module

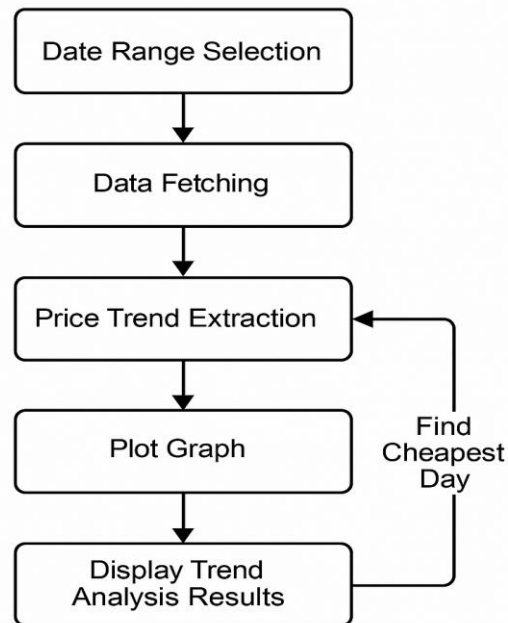


Fig 2.4 Flight Trend Analysis Module

The **Weather Report Module** provides real-time weather information for the destination city. This module enhances the user experience by offering important weather-related information that could influence travel decisions. It performs the following tasks:

1. **Weather API Integration:** The module integrates with third-party weather APIs to fetch live data on weather conditions in the destination city (e.g., temperature, precipitation, wind).
2. **Weather Alerts:** It can send alerts to users about extreme weather conditions, such as storms or adverse weather, that may affect flight plans or travel safety.
3. **User Interface:** The weather information is displayed on the frontend, alongside the flight price prediction, to provide users with a more comprehensive travel experience.

4. Adjusts the **price prediction output** accordingly.

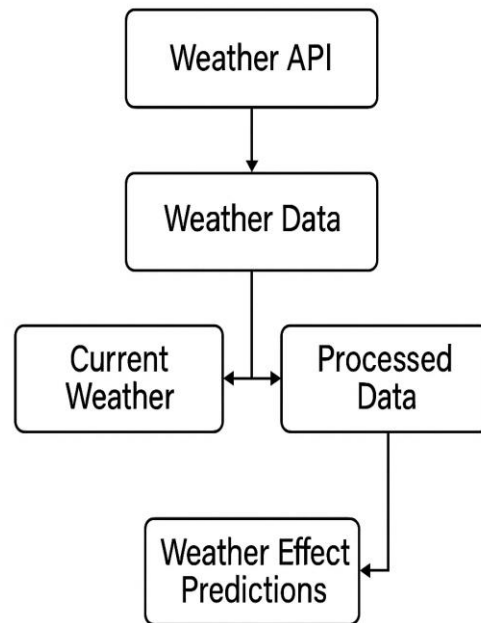


Fig 2.5 Weather Report Module

2.4 VERIFICATION TESTING FROM ACTIVITY TO MODULE LEVEL

Verification testing ensures that the software system is built correctly according to the specifications. It is done at multiple levels, starting from small units (like a function) to the full integrated system. The aim is to detect and fix errors at the earliest possible stage.

2.4.1 Levels of Verification Testing

Activity-Level :Test small parts of the code(functions or logic units)

Module-Level :Test individual modules separately

Integration-Level: Test connected components together

System-Level: Test entire application end-to-end

2.4.2 Detailed Breakdown of Verification Testing

Verification testing ensures that each component of the system works as expected, is properly integrated, and meets the specified requirements. This process is critical for ensuring the accuracy, reliability, and performance of the Flight Price Prediction System. The verification testing is divided into several levels:

Activity-Level Testing

To verify the individual functionalities within specific activities or tasks in the system. Activity-level testing ensures that each function or feature performs as intended in isolation. **Form Validation:** Verify that input fields (source, destination, start date, end date) on the frontend accept valid inputs and reject invalid ones (e.g., incorrect date format, empty fields). **Model Input Processing:** Test the data preprocessing steps (such as feature extraction and encoding) to ensure that the input data is correctly transformed before being fed into the machine learning model. **Predictions:** Test that the system returns predictions when the user submits the form and that the correct minimum price, flight details, and arrival time are displayed.

Module-Level Testing

To test each individual module independently to ensure that it functions correctly. This is a more in-depth testing approach that focuses on the logic and performance of the modules. **Data Acquisition Module:** Ensure that the dataset is loaded correctly, the data is cleaned and preprocessed, and relevant features are extracted accurately. **Price Prediction Module:** Test the Random Forest model to confirm that it predicts flight prices accurately. This includes checking that the model can handle a variety of input combinations and returns reasonable outputs. **Flight Trend Analysis Module:** Verify that the module correctly identifies flight price trends based on historical data and can generate and display trend graphs accurately. **Weather Report Module:** Test the integration with weather APIs to ensure that real-time weather data is fetched and displayed correctly for the user's destination.

Integration Testing

To verify that the modules work together as expected and that data flows seamlessly across different components of the system. **Frontend to Backend Communication:** Ensure that the HTML forms correctly submit data to the Flask backend and that the backend processes the data, runs predictions, and returns results. **Model Integration:** Verify that the Flask backend correctly loads the trained machine learning model and uses it for predictions when users input data. **Frontend Rendering:** Ensure that the prediction results, including flight details and weather information, are rendered correctly on the frontend (e.g., on `result.html` or `all_flights.html`). **Data Flow:** Test the entire flow from form submission to result display to ensure that all data is passed correctly between modules.

System-Level Testing

To verify that the entire system works as expected when all components are integrated and functioning together. System-level testing ensures that the system meets the overall requirements and works optimally in a real-world scenario. End-to-End Testing: Simulate real user interaction by entering various inputs (source, destination, start date, end date) and ensuring that the system predicts prices, displays results, and provides flight suggestions accurately. Performance Testing: Test the system's ability to handle different amounts of data and user load. Ensure that the response times for predictions are acceptable, even under heavy load. Security Testing: Verify that the system is secure and that user input is sanitized to prevent common security issues, such as SQL injection or cross-site scripting (XSS). Usability Testing: Test the user interface for ease of use and ensure that users can navigate the system without confusion. This includes ensuring that the input fields are clear and that the results are easy to interpret.

2.5 CONCLUSION:

The Flight Price Prediction System successfully demonstrates how machine learning and web development can be integrated to solve real-world problems. The modular design approach, supported by Agile methodology, ensured a smooth development process with well-defined phases and responsibilities. Each module from data acquisition and price prediction to weather reporting and trend analysis was built to function independently while contributing to the overall goal. Verification testing at all levels helped maintain the accuracy, reliability, and user-friendliness of the system.

3.SOFTWARE TESTING

3.1 INTRODUCTION

The Software Test Specification (STS) for the Flight Price Prediction using Machine Learning project provides a structured and systematic approach for validating and verifying the complete system. This document is essential to ensure that all the functional and non-functional requirements are properly met before the system is deployed for user access. In this project, users will be able to enter source location, destination location, and travel dates to

predict the expected flight ticket prices. Additionally, the system offers flight trend analysis to understand price fluctuations, alternative flight suggestions to minimize travel costs, and weather reports to help users plan trips accordingly.

Testing activities include the validation of the Data Acquisition Module, which collects and processes flight datasets; the Price Prediction Module, which uses machine learning algorithms to forecast prices; the Flight Trend Analysis Module, which displays historical and future price trends; and the Weather Report Module, which integrates external weather data based on user inputs. This document outlines the types of tests to be performed, the test cases to be designed, and the criteria for assessing the success or failure of each feature. It establishes the foundation for carrying out detailed testing by defining processes for defect detection, system performance measurement, and quality assurance to deliver a stable, accurate, and reliable application.

Key focuses of the STS include:

Verifying the accuracy of price prediction results generated by the ML model. Ensuring smooth interaction between frontend UI and backend APIs. Testing the responsiveness, usability, and performance of the user interface. Validating external integrations like weather API and flight data parsing. By adhering to the structure in this document, the Flight Price Prediction system will meet industry-grade testing standards, improving its reliability and user satisfaction.

3.1.1 Purpose

The primary purpose of this Software Test Specification is to provide a comprehensive guideline for executing tests on the Flight Price Prediction application to ensure that it functions as intended. The testing process aims to uncover any defects in the system's logic, integration, usability, or performance at an early stage of development. It specifically ensures that the flight price prediction model delivers accurate forecasts, the trend analysis reflects correct pricing behavior over selected periods, and the weather updates correspond accurately to user-selected destinations and dates.

Moreover, the testing framework validates that the user interface is easy to navigate, visually appealing, and responsive across devices. It verifies that all modules interact correctly, and that predictions are generated quickly without server delays. It also aims to test the system under various conditions such as different date ranges, different sources and destinations, and

varying internet speeds to guarantee system robustness. Overall, the purpose of this document is: To guide the testing team with detailed procedures and objectives. To reduce the risks of critical failures after deployment. To ensure the delivery of a high-quality, error-free Flight Price Prediction system that users can depend on for accurate and helpful travel planning.

3.1.2 Scope

The scope of this Software Test Specification covers all essential modules and functionalities developed in the Flight Price Prediction using Machine Learning project. The document applies to unit testing, integration testing, system testing, functional testing, non-functional testing, and user acceptance testing. The modules under test include: Data Acquisition Module: Responsible for collecting flight schedule and pricing data for model training and real-time prediction. Price Prediction Module: Uses machine learning models (such as Random Forest Regressor) to predict flight ticket prices based on user inputs like source, destination, and travel dates. Flight Trend Analysis Module: Presents trend graphs and patterns to help users choose economical travel periods. Weather Report Module: Integrates weather data corresponding to the selected journey locations and dates to provide up-to-date reports. The testing process will also focus on API response validation, frontend usability checks, model output accuracy, data handling, page loading speed, and visual consistency across devices. While the main focus remains on the internal project modules, the scope will also briefly cover external dependencies like weather APIs to ensure they are reliably integrated but will not include testing the functionality of external third-party services themselves. Furthermore, security, scalability, and performance aspects will be included within non-functional testing parameters.

The scope is intentionally designed to validate:

- Correctness of predictions
- System stability under different loads
- Ease of use for different types of users
- Consistency and accuracy in results presented

3.2 TEST STRATEGY

The Test Strategy for the Flight Price Prediction using Machine Learning project outlines the structured approach that will be followed to ensure the system is tested thoroughly and efficiently. Testing will be performed systematically at different stages to verify that each

module meets its functional requirements and performance expectations. The strategy focuses primarily on: Verifying functional correctness by comparing predicted flight prices against expected results. Ensuring data consistency when handling different combinations of source, destination, and date inputs. Validating the frontend-backend integration through proper API communication and error handling. Assessing the system's response time and load handling under varying numbers of concurrent users. Confirming that weather report integration delivers correct and timely information.

Testing will be divided into manual testing for UI elements and automated testing wherever possible, especially for repeated prediction scenarios. Different types of testing, including unit testing, integration testing, system testing, functional testing, and acceptance testing, will be adopted to cover the system comprehensively. Additionally, a risk-based testing approach will be applied, giving priority to modules that are more critical for user experience, such as the Price Prediction Module and Flight Trend Analysis Module. This ensures that critical functionalities are stable, while lesser-priority modules are tested afterward. The final goal of the test strategy is to detect and fix as many defects as early as possible, reduce the cost of error correction, and ensure the system is reliable, accurate, and user-friendly upon deployment.

3.2.1 Testing Levels

Testing in this project will be executed across multiple levels to validate both individual components and the entire system holistically. Each level focuses on different aspects of the Flight Price Prediction system:

- Activity-Level Testing:** At this initial stage, specific tasks such as data input validation, price calculation, trend graph generation, and weather API fetch are tested individually to verify correctness at the functional unit level.
- Module-Level Testing:** In this stage, modules such as Data Acquisition, Price Prediction, Flight Trend Analysis, and Weather Report are tested independently to ensure that each one behaves correctly as a standalone component. This confirms that the logic and internal flow of each module are flawless before integrating with others.
- Integration Testing:** After individual module testing, the focus shifts to inter-module communication and data flow. This level tests how well the frontend connects with the backend and how the machine learning model integrates with user interfaces to display results properly.
- System-Level Testing:** This is the final level where the entire Flight Price Prediction application is tested as a whole, including all

modules, APIs, user inputs, predictions, trend analysis, and weather data. System-level testing ensures the full application meets the user expectations in real-world usage conditions.

Each testing level is essential to gradually build confidence in the system's correctness, performance, and quality, ensuring that the final deployment is robust, accurate, and ready for user adoption.

3.2.2 Testing Methods

The Testing Methods applied for the Flight Price Prediction using Machine Learning project are carefully selected to ensure the system's functionality, performance, and reliability meet expectations. Different techniques will be combined based on the nature of each module and the project requirements. The main methods used are: Manual Testing: Manual testing will be performed for verifying the user interface, checking the form validation (source, destination, start date, end date), and ensuring that the user experience is smooth. Testers will simulate real-world scenarios by entering different combinations of inputs and verifying the predicted prices, trends, and weather updates displayed. Automated Testing: Automated scripts will be used to test backend APIs such as the price prediction API, weather report API, and flight trend analysis API. Automation ensures that the system responds correctly and consistently to repeated requests, saving time compared to manual execution for repetitive tasks.

Unit Testing: Small pieces of code like individual Python functions (e.g., date extraction, price calculation) will be tested in isolation. Each function will be checked for correctness before integrating it into the larger system, ensuring that low-level errors are caught early. Integration Testing: After unit testing, the system's different modules (frontend forms, Flask backend, ML model, and APIs) will be integrated and tested together. Special focus will be given to data passing between frontend and backend, ML model outputs, and display correctness on the frontend. System Testing: A complete end-to-end test will be conducted by simulating actual user behavior. Testers will start from entering inputs in the homepage, triggering the price prediction, viewing the detailed results, and receiving real-time weather alerts to ensure that the entire application works as intended. Performance Testing: System performance under heavy load conditions will be checked by simulating multiple users accessing the site simultaneously. The response time of predictions, loading speed of trend graphs, and weather report fetching time will be monitored to ensure no major lags. The combination of manual and automated testing, along with unit, integration, system, and

performance testing, will ensure the Flight Price Prediction System is well-tested, efficient, and ready for real-world use.

3.3 TEST OBJECTIVES

The Test Objectives define what needs to be verified during the testing phase to ensure the system is working properly. These objectives help in identifying defects early and delivering a reliable, efficient, and user-friendly system. Testing focuses on both functional and non-functional aspects to achieve a high-quality product that meets user expectations.

3.3.1 Functional Testing Objectives

The Functional Testing Objectives concentrate on verifying that each function of the system behaves exactly as expected. The focus is on correctness, accuracy, and user interaction. To verify that the input forms correctly capture the Source, Destination, Start Date, and End Date without errors. To ensure that the Predict Price feature triggers the ML model accurately and provides the minimum predicted price along with matching flight details. To confirm that the alternative flights and best date suggestions are properly displayed based on user input. To test that the weather alert system fetches and displays real-time weather updates relevant to the selected routes. To check that trend graphs for price fluctuations load dynamically and display correct information based on prediction data. To ensure navigation between pages like index.html, result.html, and all_flights.html works seamlessly. Functional testing mainly guarantees that all core features like price prediction, weather report, and flight trend analysis work properly and accurately.

3.3.2 Non-Functional Testing Objectives

The Non-Functional Testing Objectives focus on the performance, reliability, usability, and security of the system, apart from its functional correctness. To ensure that the response time for flight price prediction and weather report fetching is quick and efficient. To verify that the system can handle multiple users simultaneously without slowing down (Load Testing). To check that the user interface (UI) is attractive, interactive, and easy to navigate, enhancing the overall user experience. To test the compatibility of the application across various devices (laptop, mobile, tablet) and browsers (Chrome, Firefox, Edge). To ensure that error handling is performed properly (for example, if the user enters invalid data, the system should show proper error messages). To confirm that the system is secure, ensuring that no unauthorized access or misuse happens during form submissions or API usage. Non-functional

testing ensures the Flight Price Prediction system is not only functionally correct but also fast, user-friendly, and robust under different conditions.

3.4 TEST CASES

This section explains the test cases developed to validate the core modules of the Flight Price Prediction System. Each test case ensures that the system behaves as expected under different conditions. The testing mainly focuses on checking accuracy, handling errors properly, and providing a smooth user experience across the application.

3.4.1 Data Acquisition Module

The Data Acquisition Module collects user inputs like Source, Destination, Start Date, and End Date. It ensures valid data entry, error handling, and correct transfer of inputs to the backend for prediction.

Test Case 1: Input Field Validation

Steps:

1. Open the input form on the website.
2. Enter invalid characters like symbols or numbers in Source and Destination fields.
3. Submit the form.

Expected Result:

The system should show validation error messages and prevent form submission.

Test Case 2: Handling Missing Entries

Steps:

1. Leave some fields blank.
2. Click the "Predict" button.

Expected Result:

The system should prompt the user to fill all required fields before submission.

Test Case 3: Backend Data Transfer

Steps:

1. Enter valid Source, Destination, Start Date, and End Date.
2. Submit the form.

3. Check backend logs or network request.

Expected Result:

Data should be passed correctly to the backend in proper format without missing fields.

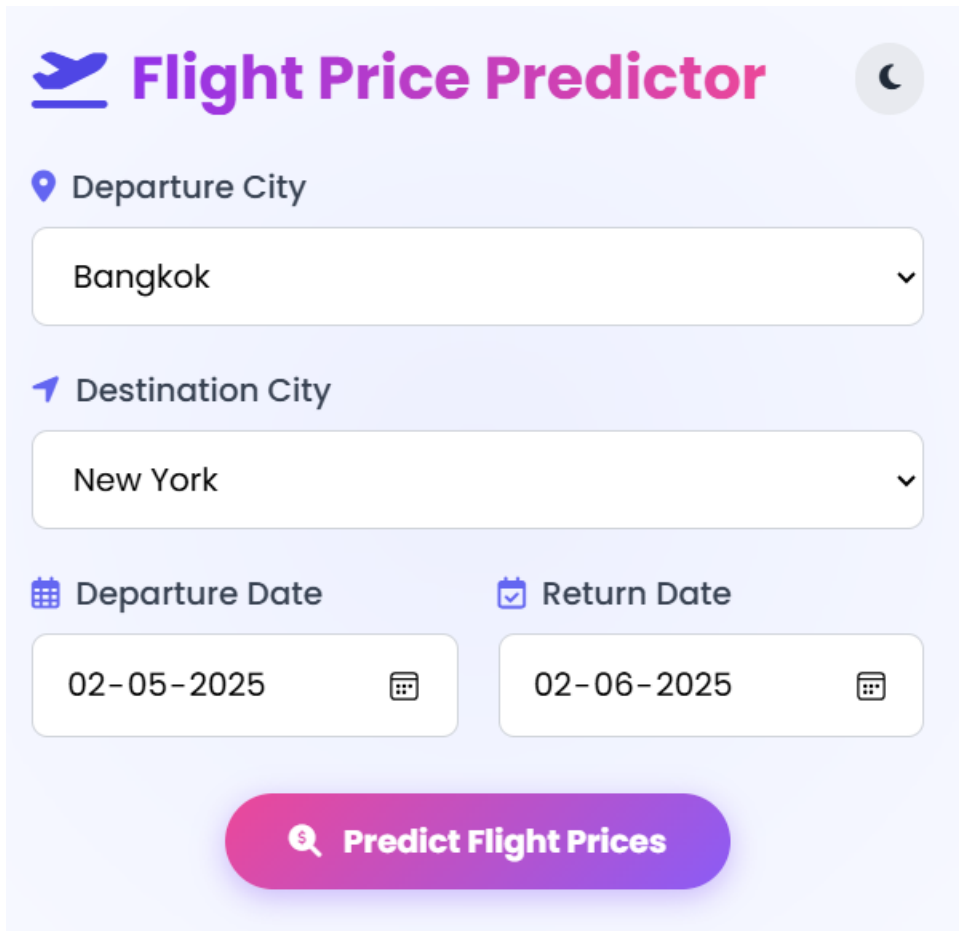
A screenshot of a web application titled "Flight Price Predictor". The form has a light blue background. At the top left is a purple airplane icon and the title "Flight Price Predictor" in purple. At the top right is a dark grey circle with a white crescent moon icon. Below the title, there are four input fields. The first is "Departure City" with a location pin icon, containing the text "Bangkok". The second is "Destination City" with a location pin icon, containing the text "New York". The third is "Departure Date" with a calendar icon, containing the text "02-05-2025". The fourth is "Return Date" with a calendar icon, containing the text "02-06-2025". At the bottom is a large purple button with a white magnifying glass icon and the text "Predict Flight Prices".

Fig. 3.1 Data Acquisition Module

3.4.2 Price Prediction Module

The Price Prediction Module processes the inputs and uses the Machine Learning model to predict the most likely flight price based on given Source, Destination, and Journey Date.

Test Case 1: Model Input Validation

Steps:

1. Submit Source, Destination, and Start Date.
2. Check if the model receives the input without any missing or incorrect fields.

Expected Result:

Model should accept correctly formatted inputs and prepare for prediction.

Test Case 2: Prediction Output Check

Steps:

1. Submit valid user input for Source, Destination, and Start Date.
2. Trigger the prediction function in the backend.

Expected Result:

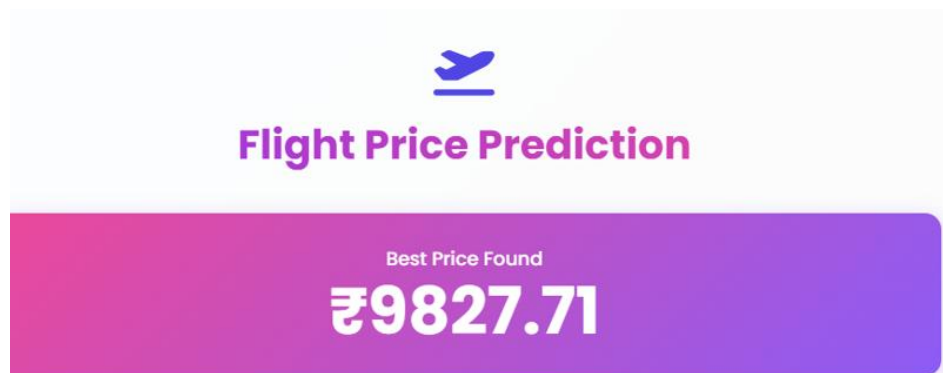
The system should generate a price prediction and return a numeric value without errors.

Test Case 3: Error Handling for Wrong Inputs

Steps:

1. Enter invalid dates (e.g., past years far back like 2000).
2. Submit the form.

Expected Result:



The system should handle such cases gracefully and alert the user without crashing.

Fig.3.2 Price Prediction Module

3.4.3 Flight Trend Analysis Module

The Flight Trend Analysis Module provides historical and future trend data of flight prices to suggest better booking dates or cheaper options.

Test Case 1: Fetching Trend Data

Steps:

1. Input valid Source and Destination.
2. Request trend analysis.

Expected Result: System should display a graph or table showing price variations over days or months.

Test Case 2: Trend Data Accuracy

Steps:

1. Compare displayed trend data against manually verified datasets.
2. Analyze for correctness.

Expected Result: Graphical trends should match the historical data pattern within acceptable range.

Test Case 3: User Interaction with Graph

Steps:

1. Hover or click on the trend graph data points.
2. Observe interactions.

Expected Result: Tooltip or information about price and date should appear smoothly without delay.

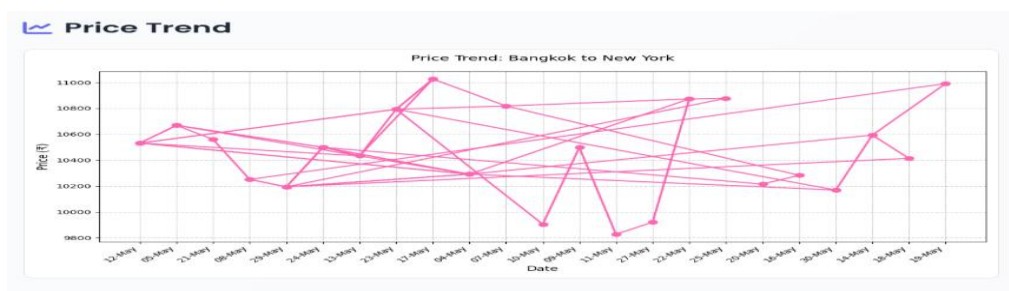


Fig 3.3 Flight Trend Analysis Module

3.4.4 Weather Report Module

The Weather Report Module fetches real-time weather updates for the selected destination, helping users to plan better.

Test Case 1: API Connection

Steps:

1. Submit Destination from the form.
2. System calls the weather API.

Expected Result:

The system should successfully connect to the weather API and fetch current weather.

Test Case 2: Weather Display Validation

Steps:

1. Get weather data from the API.
2. Display it alongside flight prediction results.

Expected Result:

Weather information like temperature, rain chance, or delays should appear clearly.

Test Case 3: Handling API Failure

Steps:

1. Simulate API downtime or error.
2. Submit a Destination.

Expected Result:

System should handle errors gracefully and show "Weather data not available" message



Fig 3.4 Weather report module

3.5.5 More Flight Details Module

The More Flight Details Module displays all available flights based on the selected Source and Destination, providing users with a complete list of flights to explore and choose

from.

Users can sort and filter flights based on Price and Date, making it easier to find flights that suit their preferences.

Features:

- Displays all flights between the selected Source and Destination.
- Sort By options:
 - Price (Low to High)
 - Price (High to Low)
 - Date (Oldest First)
 - Date (Newest First)
- Shows Flight Name, Arrival Time, Date, and Price.
- Highlights the Cheapest Flight.
- User-friendly filters to narrow down search results.

User Flow:

1. User clicks "More Flight Details" button after getting a flight prediction.
2. Redirects to all_flights.html page.
3. Displays full list of flights between selected Source and Destination.
4. User selects sorting/filtering options (Date / Price).
5. Flights update dynamically based on selection and highlight cheapest flight.

Test Cases:

Test Case 1: Flight List Display

- Steps: Predict a flight and Click More Flight Details.
- Expected Result:
All flights between the selected Source and Destination are shown with Date, Arrival, and Price.

Test Case 2: Sorting by Price

- Steps: Select "Price (Low to High)" from Sort dropdown.
- Expected Result:
Flights should reorder from the cheapest to the most expensive.

Test Case 3: Sorting by Date

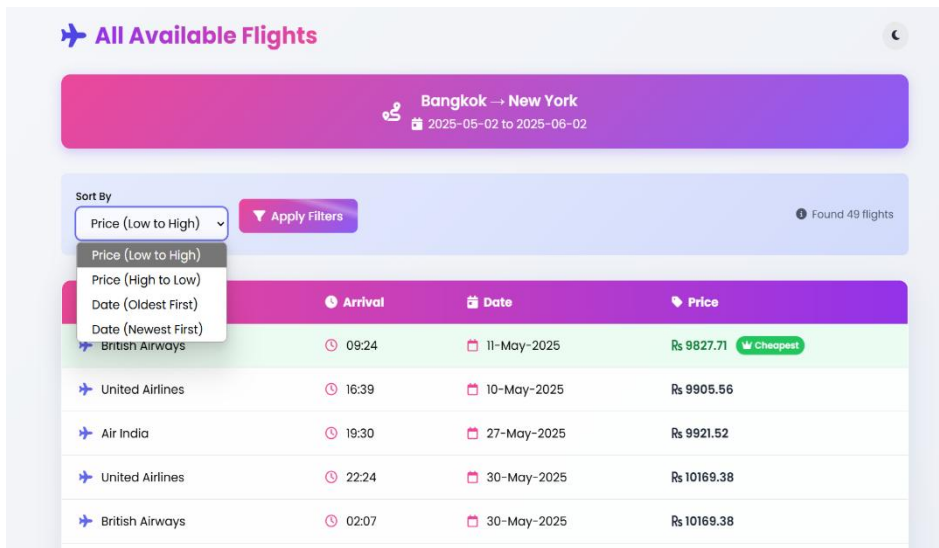
- Steps: Select "Date (Oldest First)" from Sort dropdown.
- Expected Result:
Flights should reorder starting from the earliest flight date.

Test Case 4: Cheapest Flight Highlighting

- Steps: View the flight list.
- Expected Result:
The flight with the lowest price should have a green "Cheapest" tag.

Test Case 5: No Flights Found

- Steps: Enter a Source and Destination combination with no flights.Expected Result:
A friendly message like "No flights found for selected route." should be displayed.



All Available Flights			
Bangkok → New York 2025-05-02 to 2025-06-02			
Sort By	Price (Low to High)	Apply Filters	Found 49 flights
	Price (Low to High)	Price (High to Low)	Date (Oldest First)
	Date (Newest First)	Arrival	Date
	Price		
British Airways	09:24	11-May-2025	Rs 9827.71 Cheapest
United Airlines	16:39	10-May-2025	Rs 9905.56
Air India	19:30	27-May-2025	Rs 9921.52
United Airlines	22:24	30-May-2025	Rs 10169.38
British Airways	02:07	30-May-2025	Rs 10169.38

Fig. 3.5 More Flight Details Module

3.5 TEST EXECUTION AND REPORTING

Test Execution and Reporting involves running the designed test cases on the developed system and documenting the results to ensure the Flight Price Prediction system

meets the required quality standards. This phase validates the system behavior by executing all test cases on different modules like Data Acquisition, Price Prediction, Flight Trend Analysis, and Weather Report modules. The goal is to confirm that the system delivers accurate predictions, displays real-time weather information, and provides reliable trend analysis under various input conditions.

3.5.1 Test Execution Process

The Test Execution Process ensures that all the designed test cases are run systematically on the completed modules of the project. Each test case focuses on verifying individual functionalities, handling of user input, output accuracy, and system performance. The steps involved are: Test Environment Setup: The system environment is prepared, ensuring that the Flask backend, Machine Learning model, and frontend are properly connected and operational. Running Test Cases: Each functional module like Price Prediction and Weather Reporting is tested individually first (Unit Testing) and then tested together (Integration Testing) by executing the planned test cases step-by-step. Logging Results: The outputs for each test case are observed and recorded systematically. Any deviations or unexpected behaviors are noted for later analysis. During execution, special attention is given to validating price accuracy, weather data correctness, error handling for invalid inputs, and graphical trend display consistency.

3.5.2 Test Reporting

After executing the test cases, all results are compiled into a formal Test Report. This document provides a clear picture of how well the Flight Price Prediction system performs against expectations. The key components of the Test Report are: Summary of Executed Test Cases: A listing of all test cases that were executed, mentioning pass or fail status. Defect Identification: Any bugs, issues, or unexpected results observed during testing are listed clearly with screenshots or error messages wherever applicable. Module-wise Status: Each module's testing status (like Data Acquisition, Price Prediction, Flight Trend Analysis, Weather Report) is summarized separately. Recommendations: If any bugs are critical, necessary improvements or corrective actions are recommended before project finalization. The report ensures transparency about system reliability and serves as proof of validation before deployment.

3.5.3 Post-Testing Actions

Once all testing activities are completed and documented, certain important post-testing actions are carried out to ensure project readiness for real-world use. The major post-testing activities include: Bug Fixing and Retesting: Any bugs identified during testing are fixed by the development team, and the affected modules are retested to confirm the issues are resolved. System Optimization: Minor performance improvements are applied based on the testing feedback, such as optimizing prediction speed or loading time of trend graphs. Final Validation: A final round of testing is performed after all fixes, ensuring the complete system works perfectly without any critical issues. Deployment Preparation: After successful validation, the system is packaged and prepared for final deployment in the target environment. These actions ensure the Flight Price Prediction system is stable, reliable, and ready for practical use by end-users.

3.6 TEST MAINTENANCE

Test maintenance ensures that the testing process remains relevant and effective throughout the development lifecycle. As the project evolves, both the system and the testing strategies need continuous updates to ensure the reliability, accuracy, and stability of the application. This process helps identify defects early and provides feedback on the overall quality of the product. Effective test maintenance is essential for ensuring the system continues to meet the expectations of its users, particularly as new features are integrated or bugs are fixed. In the case of your **Flight Price Prediction system**, test maintenance becomes increasingly important as the machine learning model improves, additional features are added, and the system is scaled to handle more users and data. Each update or change made to the system should prompt a review and possible revision of the test cases, ensuring that the integrity of the application is upheld.

3.6.1 Test Case Updates

Test cases must be regularly updated to ensure they remain relevant as the Flight Price Prediction system evolves. This includes modifying test cases whenever there are feature updates, bug fixes, or changes in the underlying system. Add New Test Cases: Whenever new functionality is introduced (e.g., new flight prediction features, alternative flight suggestions), new test cases must be created to verify these features. Modify Existing Test Cases: Update test cases when existing features are altered, such as changes to the model, data structure, or user interface. Handle Edge Cases: Continuously assess and include edge cases, such as invalid dates, unexpected input data, or boundary conditions, to ensure the system performs well under

all scenarios. Regular Review: Ensure that all test cases reflect the current state of the system and that obsolete tests are removed or updated.

3.6.2 Automated Test Maintenance

Automated tests help to efficiently validate system functionality and detect regressions. However, automated tests need ongoing maintenance as the system undergoes updates. Update Test Scripts: After any change, such as a new feature or backend modification, update the test scripts to ensure they are aligned with the latest system behavior. Test Coverage: Ensure that all critical functionalities (e.g., price predictions, flight data retrieval) are covered by automated tests. Regression Automation: Use automated regression tests to check that previous functionality still works correctly after updates or fixes. Optimize for Performance: As the system scales, optimize automated tests to ensure they run efficiently, without unnecessary delays or resource consumption.

3.6.3 Regression Testing

Regression testing ensures that updates to the **Flight Price Prediction system** do not break existing functionality. It is crucial after each update or new feature addition to verify the integrity of the overall system.

- **Comprehensive Testing:** Run all critical test cases, including previous ones, to ensure that existing features (e.g., price prediction, flight information display) are unaffected by new changes.
- **Automated Regression Suite:** Implement an automated regression suite that can be executed after every change, ensuring fast and consistent validation of the system's stability.
- **Manual Testing for Complex Scenarios:** For complex use cases or scenarios that may not be easily automated, conduct manual testing to ensure the system behaves as expected.
- **Frequent Testing:** Perform regression testing regularly, especially after any modification to the machine learning model or when integrating new features like real-time data updates or additional flight information.

3.7 ACCEPTANCE CRITERIA

Acceptance criteria define the conditions under which the **Flight Price Prediction system** is considered complete and fully functional. These criteria encompass both functional and non-functional requirements, as well as specific benchmarks for the machine learning model's accuracy and the seamless integration between the user interface and backend. The acceptance criteria ensure that all system components, from the frontend to the backend and the machine learning model, perform as expected and meet the business needs.

3.7.1 Functional Requirements

Functional requirements outline the core features and functionalities the system must provide to meet user needs and expectations. For the **Flight Price Prediction system**, this includes:

- **Price Prediction:** The system must accurately predict flight prices based on user inputs like source, destination, start date, and end date.
- **Flight Details Display:** After a prediction, users should see essential details such as flight name, departure/arrival times, and prices within the selected date range.
- **Alternative Flight Suggestions:** If available, the system should suggest alternative flights based on lower prices or better timings.
- **Real-Time Weather Alerts:** The system must provide real-time weather updates related to the selected flight's route.
- **Flight Trend Analysis:** Display flight price trends over time, indicating how prices fluctuate for specific routes or dates.
- **Interactive User Interface:** The frontend must allow users to easily input their flight criteria and interact with the results (e.g., view, sort, or filter).

3.7.2 Non-Functional Requirements

Non-functional requirements define the quality and performance criteria for the **Flight Price Prediction system**. These include:

- **Performance:** The system should be able to handle multiple users making concurrent flight searches without significant delay or system crashes.
- **Scalability:** As the number of users and available flights increases, the system should scale to handle more data and users efficiently.

- **Security:** Sensitive data such as user information and payment details must be securely stored and transmitted, adhering to industry standards for data protection.
- **Usability:** The system should provide a user-friendly experience with intuitive navigation, clear instructions, and responsive design, ensuring that users can easily input flight details and understand the results.
- **Reliability:** The system should operate without errors or crashes, ensuring that users receive consistent, accurate results each time.

3.7.3 Machine Learning Model Accuracy

The accuracy of the machine learning model is a critical aspect of the **Flight Price Prediction system**. For the model to be considered successful, the following criteria should be met:

- **Accuracy Metric:** The model must achieve a minimum accuracy rate for predicting flight prices within a reasonable error margin (e.g., less than 10% error in price predictions).
- **Feature Handling:** The model should accurately handle the four key input features: source, destination, start date, and end date, and produce reliable price predictions.
- **Continuous Learning:** The model must be able to be retrained with updated data periodically, improving accuracy and adapting to changes in the flight market.
- **Real-Time Predictions:** The model should be capable of generating flight price predictions in real-time as users input their data into the system.

3.7.4 User Interface and Backend Integration

The integration between the **Flight Price Prediction system's** user interface and backend is crucial for delivering a seamless user experience. The following criteria must be satisfied:

- **Data Flow:** The UI must send user inputs (source, destination, dates) to the backend for processing. The backend should process these inputs, pass them to the machine learning model, and return the prediction results to the frontend.

- **Error Handling:** Any errors, such as invalid input or failed predictions, must be properly handled by displaying user-friendly error messages and ensuring the system continues functioning smoothly.
- **Performance:** The frontend and backend integration should ensure quick data exchange, so users can receive results without significant delays, especially during peak usage times.
- **Consistency:** The frontend should reflect the data and results accurately, matching the information returned by the backend. This ensures that users see correct and up-to-date flight prices, details, and predictions.

3.8 FINAL ASSESSMENT

The **Final Assessment** of the **Flight Price Prediction system** involves evaluating its overall performance, usability, and meeting the specified acceptance criteria. This stage includes validating the system's accuracy in predicting flight prices, ensuring that all functionalities work seamlessly, and confirming that the system meets both functional and non-functional requirements.

Key aspects of the final assessment include:

- **Machine Learning Model Validation:** The performance of the machine learning model is reviewed to ensure that it provides accurate price predictions. The model's predictive accuracy should meet the predefined threshold (e.g., less than 10% error) and handle all input features appropriately.
- **System Integration:** The integration between the frontend, backend, and the machine learning model should be thoroughly tested to ensure smooth data flow. The system must operate as a unified solution, providing accurate predictions based on user inputs and displaying them effectively on the frontend.
- **User Experience Testing:** Usability testing should confirm that the system is user-friendly and provides an intuitive experience. This includes evaluating the layout, responsiveness, and ease of interaction with the input forms, buttons, and results display.

- **Performance and Scalability:** The system should be assessed under various usage conditions to test its ability to handle multiple requests, ensuring that it operates efficiently without degradation in performance.
- **Security and Reliability:** The system's security measures should be assessed, particularly in terms of data protection (e.g., user information and payment data). The system should also be evaluated for reliability, ensuring it can run for extended periods without errors or crashes.

3.9 CONCLUSION

The **Flight Price Prediction system** has successfully implemented a machine learning-based solution that provides accurate flight price predictions based on user inputs. By carefully addressing both functional and non-functional requirements, the system delivers a comprehensive, user-friendly tool that helps travelers predict flight prices, view relevant flight details, and explore alternative options. Throughout the development, the system has been designed to prioritize accuracy, ease of use, and scalability.

Key aspects that were successfully integrated into the system include accurate price predictions powered by machine learning, ensuring that predicted prices are within a reasonable error margin. The user interface has been optimized for seamless navigation, allowing users to interact with the system effortlessly and retrieve necessary information quickly. Additionally, the system's performance has been carefully tested to ensure it can handle multiple simultaneous users without any degradation, offering a smooth experience even during high-traffic times.

Looking forward, the system has great potential for future enhancements. With its current scalability, it is positioned well for additional features like personalized flight recommendations, real-time flight price updates, and advanced data analytics for further optimization. Overall, the **Flight Price Prediction system** represents a robust and scalable solution for helping users make informed decisions when booking flights, and with continuous improvements, it can evolve into an even more powerful tool.

GLOSSARY

- **Machine Learning (ML)** :A branch of artificial intelligence (AI) where systems learn from data patterns and make decisions without being explicitly programmed.
- **Flight Price Prediction:** The process of estimating future flight ticket prices based on historical data and influencing factors using machine learning techniques.
- **Dataset:** A structured collection of data used to train and test machine learning models. In this project, the dataset includes flight details such as source, destination, date, and price.
- **Model Training:** The phase where a machine learning model is taught to identify patterns from a dataset to make accurate predictions.
- **Flask:** A lightweight Python web framework used to create the backend server for hosting the machine learning model and APIs.
- **API (Application Programming Interface):** A set of rules that allows communication between the backend (Flask server) and the frontend (web interface).
- **Frontend:** The part of the application that users interact with directly, developed using HTML, CSS, and JavaScript.
- **Backend:** The server-side part of the application that processes user input, runs the ML model, and sends responses to the frontend.
- **Feature Extraction:** The process of selecting important attributes (like Source, Destination, Journey Date) from the dataset for training the machine learning model.
- **Deployment:** Making the developed machine learning model and web application accessible to users via a hosted environment.
- **Regression Algorithm:** A type of machine learning algorithm used to predict continuous values, such as flight prices. Example: Random Forest Regressor used in this project.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.
- [2] S. Raschka and V. Mirjalili, Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2, 3rd ed., Packt Publishing, 2019.
- [3] M. Lantz, Machine Learning with Python for Everyone, Addison-Wesley Professional, 2019.
- [4] F. Chollet, Deep Learning with Python, 2nd ed., Manning Publications, 2021.
- [5] “Flask Documentation,” [Online]. Available: <https://flask.palletsprojects.com/>
- [6] “Scikit-learn: Machine Learning in Python,” [Online]. Available: <https://scikit-learn.org/>
- [7] “Kaggle Dataset: Flight Price Prediction,” [Online]. Available: <https://www.kaggle.com/datasets/nikhilmittal/flight-fare-prediction-mh>
- [8] “HTML Living Standard,” WHATWG, 2025. [Online]. Available: <https://html.spec.whatwg.org/>
- [9] “JavaScript MDN Web Docs,” Mozilla Foundation, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>